

WaveQL: A Joined Debug Store for Agentic Bug Repair on an Out-of-Order RISC-V Core

Jayaraj Jayakumar

University of California, Santa Cruz

Santa Cruz, CA, USA

jj8@ucsc.edu

A³ CHIA Hackathon · Track 1: discovery and resolution of bugs in open-source designs such as BOOM

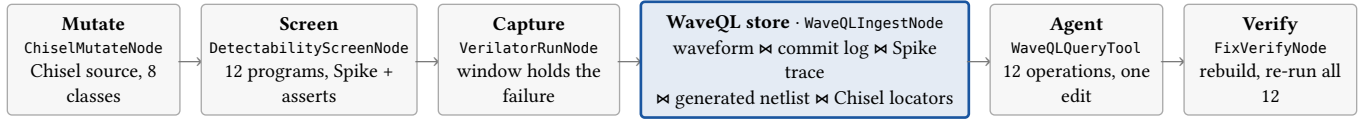


Figure 1: The loop as a CHIA graph. Each box is a CHIA block; CHIA’s own Chipyard nodes build and run the core. The agent never sees a raw dump, and only a rebuilt core that passes every program in lockstep with Spike counts as repaired.

Abstract

When a processor fails a test, an AI agent is handed logs. WaveQL hands it what a hardware engineer would want: the waveform, the core’s commit log and the golden Spike trace joined on cycle in one queryable store, with a causal walk over the generated netlist that answers *why* a signal held a value — built as CHIA blocks around BOOM, the out-of-order RISC-V core. To measure it we built BuggyBOOM, a verified-repair benchmark: 111 bugs in BOOM’s Chisel source, from 248 screened mutants, where a repair counts only if the rebuilt core passes all twelve programs in lockstep with Spike. Across 669 scored episodes of gemini-3.1-pro, the joined view repairs a statistically indistinguishable share of bugs (17.9% against 21.7% for the same evidence as text, difference -3.8 points, 95% CI [-8.8, +1.1]) while reading $0.67\times$ the tokens per episode (95% CI [0.6, 0.75]) and 19% fewer per verified repair. The limit is navigation, not evidence: with the store, the agent spent its turns searching 29,183 signal names and proposed a patch in 57.2% of episodes against 70.7% once its patch was in the right file, it repaired at a similar rate. Scale changed the answer: on the first 11 tasks the joined view led by +10.6 points, a lead that is gone at 111. The whole study — 426 patches, each verified by a full rebuild of the core — ran as one CHIA loop on four 96-core cloud VMs for about \$913.

Keywords

agentic hardware debugging, waveforms, co-simulation, RISC-V, BOOM, CHIA

1 Introduction

When a chip design fails a test, a person opens a waveform viewer and scrolls. An AI agent has no screen; it gets raw logs and a first-mismatch report. For an out-of-order core the gap is widest: the instruction that commits a wrong value may have been corrupted dozens of cycles earlier, in another unit, by a signal that never appears in any log. Debugging it means joining three records — what the golden model expected, what the core committed, what every signal held — on the one key they share, the cycle. CHIA’s CosimNode computes the first architectural divergence against Spike and then discards it into a text window; agent-facing waveform tools compare waveform against waveform, which needs a golden waveform that does not exist when the bug is real.

WaveQL puts the join in the loop, and BuggyBOOM measures what it is worth on the hardest open-source target CHIA supports: BOOM v3, an out-of-order RISC-V core, repaired at the Chisel source and verified by rebuilding the core.

- **A verified-repair benchmark on an out-of-order core.** 111 bugs in BOOM’s Chisel source, observed failing by Spike lockstep (46) or by a fired Chisel assertion (65), from 248 mutants in eight operator classes; a repair counts only if the rebuilt core passes all twelve programs (§3).
- **A join that is exact, and a walk that explains.** Keyed on the core’s own cycle counter, the store reproduces BOOM’s commit log 235/235; the netlist evaluator reproduces simulator values at 97.7%, and why follows a wrong value back through the ROB’s SRAM macro to the D-cache (§2, §4).
- **Comparable repairs for a third fewer tokens.** With the store the agent read $0.67\times$ the tokens per episode and 19% fewer per verified repair, at a repair rate statistically indistinguishable from the text control’s (§5).
- **Navigation, not evidence, is the bottleneck.** The agent with the store spent its budget finding its way around 29,183 signal names, and proposed a patch in 57.2% of episodes against 70.7% (§6).
- **Scale changes the answer.** An early +10.6-point lead on 11 tasks vanished at 111 the same CHIA loop running across four cloud VMs is what turned an anecdote into a measurement (§5, §8).

For CHIA this makes debugging collateral a first-class loop input. WaveQLIngestNode and WaveQLQueryTool give any loop a joined, cycle-indexed view of a failing run; FixVerifyNode makes “only a rebuilt, Spike-clean core counts” a node; and BuggyBOOM gives the bug-resolution track a benchmark on BOOM whose every score is machine-verified.

2 The loop and the store

The loop (Figure 1) mutates BOOM’s Chisel, builds and screens each mutant with CHIA’s Chipyard nodes, records a waveform window around the failure, ingests it, hands the agent a query surface, and verifies whatever it proposes. **WaveQL** streams the Verilator VCD through pywellen into DuckDB, alongside the commit log, the Spike trace, the divergence, a signal-to-Chisel map built from firtool’s locators, and a query log. Every row is keyed on the core’s own cycle counter, the key that makes the join exact. Answers are *asof*, since a waveform stores changes and a range filter returns nothing for a signal holding its value; every answer is size-capped and logged, so queries per repair is a measured quantity. The surface is twelve operations (Table 1); four answer what a recording cannot: `stall_report` (why the pipeline stopped retiring), `source_of` and `drivers` (which Chisel line and which

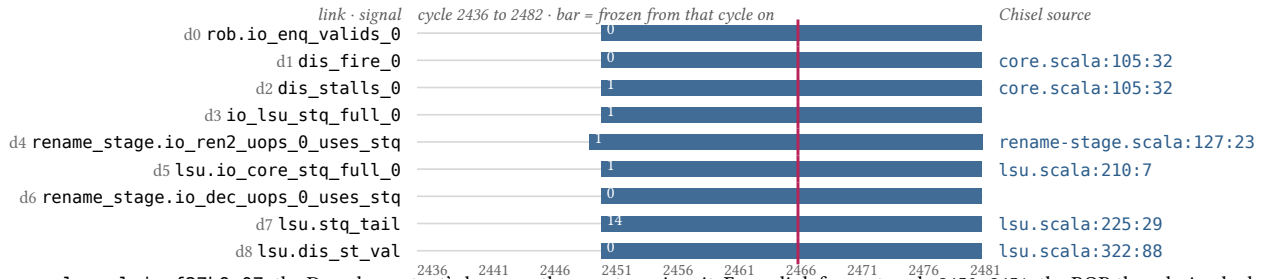


Figure 2: waveql-explain f27b9c07: the D-cache mutant’s hang, as the agent receives it. Every link froze at cycle 2450–2451; the ROB then drained what was in flight, the last instruction retired at **cycle 2467 (magenta)**, and nothing retired for 8193 cycles after. Reading down: the ROB is empty because dispatch stalled because the store queue is full. The defect is in `dcache.scala`, behind the LSU.

signals produce a value), and why (which operands *account for* the value a signal held).

Table 1: The WaveQL query surface. Every answer is size-capped and logged.

operation	answers
first_divergence	how and when the run failed: the first mismatch with Spike, or the assertion that fired
stall_report	why retirement stopped: last retired cycle, ROB head, the chain of stalled signals
why	which operands account for a signal’s value, walked back through the netlist
drivers	the signals a value is computed from, across module boundaries
source_of	the Chisel file and line that produce a signal
commits	instructions retired around a cycle: pc, instruction, destination, value
inflight	ROB occupancy at a cycle: valid entries, head, tail
state_at	the value of any signals at a cycle
window	signal changes within a cycle range
trace_signal	one signal across a cycle range, including the value it entered with
find_signals	search among 29,183 signal names
sql	read-only SQL over the joined tables

3 BuggyBOOM: bugs an oracle sees

Mutation. A Scala lexer drives eight operator classes over BOOM v3’s Chisel source — comparison and boundary flips, `&&/||` swaps, inverted selects, handshake and pipeline-depth edits (a dropped `RegNext`), index arithmetic and constant flips — excluding sites inside assertions and the commit-log `printf` the join reads, and performance-only sites a correct-but-mispredicting core would hide. Three seeded, class-balanced draws sampled 248 distinct mutants.

Two oracles. Each mutant runs twelve programs — ISA tests, CSR and atomic tests, and benchmarks — under Spike lockstep and with BOOM’s own Chisel assertions live. The second oracle matters on an out-of-order core: a deadlocked pipeline never commits anything wrong, so Spike alone scores a hang as a survivor. 111 mutants are killed by a localizable oracle, 65 of them by an assertion.

Evidence that contains the failure. Each kill ships with its co-simulation log, commit log and a waveform window. A hang is asserted 8,192 cycles after the last commit, so its window is aimed there, and every window is checked against the cycle of the failure: 95 of 100 contain it (the rest diverge within their first two commits, before the dump’s first sample).

Table 2: BuggyBOOM by mutation class: mutants screened and killed, and the verified repair rate per task for each arm.

Class	screened	killed	tasks	waveql	control
boundary-off-by-one	30	3	3	22.2%	33.3%
comparison-inversion	35	23	23	21.7%	25%
condition-logic	35	22	22	30.3%	42.4%
constant-flip	34	12	12	13.9%	25%
handshake-protocol	35	11	11	21.2%	21.2%
index-arithmetic	9	4	4	0%	8.3%
pipeline-depth	35	12	12	6.9%	2.8%
select-inversion	35	24	24	11.1%	8.3%

4 From what to why

A waveform records what each signal was; why it took that value lives in the netlist. WaveQL parses firtool’s SystemVerilog into expressions with Chisel locators attached, resolves ports across module boundaries in both directions, and recomputes the anonymous `_GEN_*` temporaries — 76% of a typical cone, never traced by Verilator — from the registers beneath them; the evaluator reproduces the simulator’s values at 97.7% over 1,027 signals. `why(signal, cycle)` follows only the operands that account for a value, ranks links by how close to the symptom they stopped moving, and crosses the ROB’s SRAM macro by finding the write that produced the value read (Figure 2, Figure 3).

Block	Signal holding the value	Chisel
ROB	rob.io_commit_debug_wdata_1	
written at cycle 1685 by port W3 · through the ROB’s SRAM macro		
ROB	rob.io_debug_wb_wdata_0	
Writeback	ll_wbarb.io_out_bits_data	Arbiter.scala:133:7
Writeback	ll_wbarb.io_in_0_bits_data	
Execute	memExeUnit.io_ll_iresp_bits_data	execution-unit.scala:208:7
Execute	memExeUnit.io_lsu_io_iresp_bits_data	
LSU	io_lsu_exe_0_iresp_bits_data	
LSU	lsu.io_core_exe_0_iresp_bits_data	lsu.scala:210:7
LSU	lsu.io_dmem_resp_0_bits_data	
D-cache	dcache.io_lsu_resp_0_bits_data	dcache.scala:413:7

Figure 3: Mutant `c80f8bde` commits `0xffffffffdeadbeef` to `x5`; Spike expects `0x12345678`. `why` follows that exact value back from the commit point, through the ROB’s SRAM macro, the writeback arbiter, the execution unit and the LSU, to the D-cache response. Only links holding the value are shown. The defect is in `lsu.scala`.

Localization is a prior in an out-of-order core. Held out on 96 bugs, the walk names the defect’s file in 34 a fixed list of the five files it names most often (`core`, `rob`, `lsu`, ...) contains it 45 times, and another bug’s walk 27.7% of the time. Every failure reaches the same hub files within cycles, so what the waveform can add is the explanation — which signal held what, and on which Chisel line — not the file name.

5 Does the join help an agent repair?

Setup. Two arms, identical except for the evidence surface: the WaveQL arm queries the store; the control reads the same bytes the store was built from — co-simulation log, commit log, first mismatch — through capped `read/grep/tail`. Both get identical Chisel read access and one structured edit, `propose_fix`, and the same task statement, which names no file, module or cycle. Model `gemini-3.1-pro` at temperature 0, 30 turns and a 15-minute wall-clock budget per episode, three seeds per task and arm; 669 episodes are scored, and every proposed patch that applies is verified by rebuilding the core and re-running all twelve programs under Spike (Table 3).

Table 3: 669 scored episodes on 111 tasks, `gemini-3.1-pro`. Repair rates are per task, averaged over seeds, with 95% bootstrap intervals over tasks.

	WaveQL		text control	
verified repair rate (per task)	17.9%	[11.7%, 24.5%]	21.7%	[14.7%, 28.9%]
episodes repaired	60 / 334		73 / 335	
patch in the defect’s file	93 / 334		116 / 335	
tokens per episode	378k		565k	
tokens per verified repair	2,105k		2,595k	

Repairs. Paired by task, the difference in verified repair rate is -3.8 points, 95% CI [-8.8, +1.1] — statistically indistinguishable; the interval excludes a gain of ten points or more. **Cost.** Per task, the WaveQL arm read $0.67\times$ the control’s tokens per episode (95% CI [0.6, 0.75], 10,000 resamples over tasks), and spent 19% fewer tokens per verified repair. The joined view read less on 82 of 111 tasks (Figure 4), and the saving is largest where the bug is a wrong value: following a value back through the netlist replaces searching the logs for it (Table 4).

Table 4: Verified repair rate per task and tokens per episode, by how the mutant failed: Spike saw a wrong committed value or PC, or a Chisel assertion fired.

failure	tasks	WaveQL	control	tokens / episode
wrong value or PC	46	9.8%	11.1%	441k vs 744k (0.59 $\times$)
hang or assertion	65	23.6%	29.2%	333k vs 437k (0.76 $\times$)

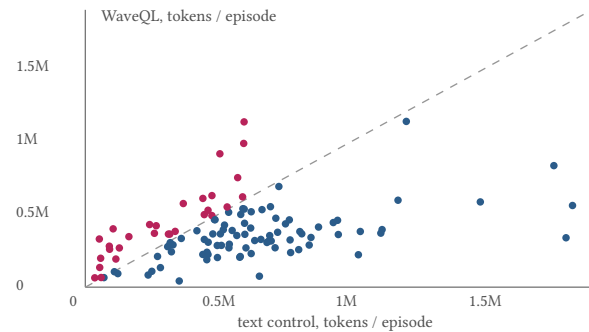


Figure 4: Mean tokens per episode for each task, WaveQL against the text control. Below the dashed diagonal (blue) the joined view read less: 82 of 111 tasks.

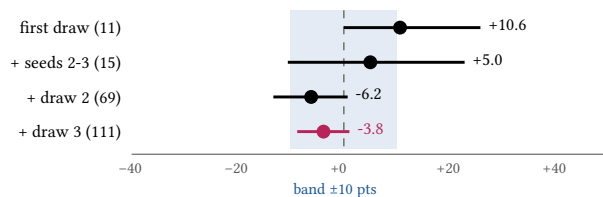


Figure 5: Scale changes the answer. WaveQL – control verified repair rate (points, 95% bootstrap interval) as the corpus grew, recomputed at each stage by the same analysis; paired tasks in brackets. Shaded: ± 10 points.

Scale. On the first 11 tasks the WaveQL arm led by +10.6 points (Figure 5). Repeated runs of one cell do not always agree — the same task, arm and seed repaired the bug once and failed once — so a dozen tasks cannot separate an effect this size from noise; 111 can. Running the loop across four cloud VMs is what made the second number affordable.

6 Where the budget goes

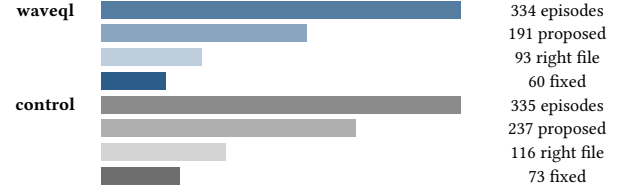


Figure 6: Where each arm’s episodes went: every scored episode, those that proposed a patch, those whose patch was in the defect’s file, and verified repairs.

The two arms diverge before the edit, not at it (Figure 6). The WaveQL agent proposed a patch in 57.2% of episodes against 70.7%, and made 30 or more tool calls in 76.6% of them against 54.9%. Its most-used operation was `find_signals` (1814 calls), a search over 29,183 signal names; why, the operation built to make that search unnecessary, was 2.8% of its calls. Once a patch was in the right file, the two arms turned 63.4% and 62.9% of them into verified repairs.

When the agent does take the short path, the chain shows what a log cannot. On a pipeline-depth mutant that dropped one `RegNext` from the exception-return path, both arms reached `core.scala`. The control knew the redirect PC was wrong and edited the line computing it; the chain showed the value arriving one cycle out of step with the flush, and the WaveQL arm restored the missing stage — the core then passed all twelve programs. A log says where a failure surfaced; a causal chain says what a signal held and when it stopped. The next surface should lead with that chain rather than wait to be asked for it.

7 One repair, end to end

Table 5: A WaveQL episode that repaired a BOOM bug: the injected edit flipped the comparison that ends the writeback unit’s multibeat release, and the agent’s patch is its exact inverse. 19 queries, from the transcripts in the artifact.

step	what the agent asked, and what it learned
first_divergence	a TileLink monitor assertion at cycle 2469: ‘C’ channel opcode changed within a multibeat operation
10 queries	<code>find_signals</code> , <code>window</code> , <code>trace_signal</code> , <code>state_at</code> : the D-cache’s C channel, which carries releases to L2
why	<code>dcache.auto_out_c_valid</code> $\leftarrow$ <code>nodeOut_c_valid</code> (<code>dcache.scala:431</code>) $\leftarrow$ the release arbiter’s state $\leftarrow$ the writeback unit’s <code>release.valid</code>
read_source	three reads of <code>dcache.scala</code> , around the writeback unit’s state machine
propose_fix	<code>dcache.scala:132: data_req_cnt != (refillCycles-1).U</code> $\rightarrow$ <code>===</code>
FixVerifyNode	rebuilt core passes 12/12 programs in lockstep with Spike

The mutant flipped one comparison in the D-cache’s writeback unit, so it ended a multi-beat release to L2 one beat early — a bug that never commits a wrong value, only violates the bus protocol, and that only the TileLink monitor on the bus catches. From the

assertion the agent found the channel, asked why its valid signal held, followed the chain through the release arbiter into the write-back unit, read that state machine, and restored the comparison (Table 5). A log names the monitor; the chain names the state machine behind it.

8 The loop at scale

The study ran as one CHIA loop across four 96-vCPU VMs, each running sixteen Chipyard worker checkouts; screening a draw of about 100 mutants takes an hour and a half on one VM. Agent episodes run one process per task, since the tools’ netlist evaluation is CPU-bound. Verification — a full Chisel-to-Verilator rebuild per patch — is the expensive stage, so verifiers on every VM draw from one shared queue and claim each cell by creating an object in Cloud Storage with `ifGenerationMatch=0`, which succeeds for exactly one caller: every patch is verified exactly once. Every recorded build failure is traced to the file its own patch edited, and a stopped verifier restores both the mutation and the agent’s patch, so no verdict can be charged to another cell’s residue (Table 6).

Table 6: The study as one CHIA loop.

Chisel mutants screened, three draws	248
bugs killed by Spike lockstep or an assertion	111
agent episodes scored, two arms $\times$ three seeds	669
patches verified by a full rebuild of the core	426
waveform windows that contain their failure	95 / 100
VM-hours (four 96-vCPU VMs)	12
cost: model tokens + cloud compute	\$863 + \$50

9 What this gives CHIA

The track asks for the discovery and resolution of bugs in widely used open-source designs such as BOOM. WaveQL does it on BOOM itself, end to end: from a Chisel edit to a machine-verified repair, every step a CHIA block. Three pieces carry over to any CHIA loop. `WaveQLIngestNode` and `WaveQLQueryTool` turn a failing run into a joined, cycle-indexed store any agent can query, with the divergence, the waveform and the source in one place. `FixVerifyNode` makes a rebuilt, Spike-clean core the only evidence of a repair. BuggyBOOM gives the community 111 verified tasks on an out-of-order core, per class, with the evidence each kill ships with.

Lessons for agentic debugging tools. Put the causal answer first: the operation that explains a value was 2.8% of the agent’s calls, yet it is the step that located the repair of §7. Budget the navigation: a richer surface spent turns the text agent spent editing (§6). Measure at scale: a dozen tasks showed a +10.6-point lead that 111 did not (§5).

10 Related work

`wave-mcp` [1] gives agents dozens of waveform tools, including waveform-to-waveform first divergence, which needs a golden waveform. BluesFL [2] localizes faults in an in-order RISC-V core from co-simulation divergence and per-signal waveform reads. WaveQL differs in the query surface (a joined, cycle-indexed store with a causal netlist walk), the target (an out-of-order core) and the outcome (verified repair rather than localization). HWE-Bench [3] contains repository-level hardware repair tasks; Encarsia [4] injects BOOM bugs at the RTLIL level and proves their observ-

ability. BuggyBOOM contributes Chisel-source mutants of BOOM with a rebuild-and-lockstep repair oracle. We build on BOOM [5], Chipyard [6], CHIA [7] and Spike [8].

11 Scope

The results are on one core (BOOM v3, MediumBoom), one model family (Gemini; `gemini-2.5-pro` repaired none of 8 episodes in either arm, so the study uses `gemini-3.1-pro`) and injected bugs. Verilator is two-state, hiding X-propagation. Six mutants were drawn in two of the three draws and count once.

12 Released as CHIA blocks; reproducibility

The loop is five CHIA blocks — `ChiselMutateNode`, `DetectabilityScreenNode`, `WaveQLIngestNode`, `WaveQLQueryTool`, `FixVerifyNode` — composed by `waveql-loop`, and each is usable alone. The artifact, at github.com/jyrj/WaveQL, contains the loop, the store and query surface, the mutator, the screening and repair harness, the cloud scale-out scripts, BuggyBOOM’s manifests, every measurement file cited and the transcripts of 667 of the 669 scored episodes; every number and figure in this paper is generated from those files by a script. `waveql-explain f27b9c07` reproduces Figure 2.

Acknowledgement of AI assistance. Claude (Anthropic) was used extensively throughout this work: writing and refactoring code, running and analysing the measurement campaigns, and drafting and revising this paper. Gemini models are the agents under test. All experimental design decisions, all measurements reported here, and the final content and framing of this paper are the responsibility of the human author.

References. [1] Tencent, `wave-mcp`: waveform tools for agents over MCP, 2025. [2] BluesFL: fault localization for RISC-V cores from co-simulation divergence, DAC 2026. [3] HWE-Bench: repository-level hardware bug repair, 2025. [4] Encarsia: provably observable bug injection for BOOM at RTLIL level, USENIX Security 2025. [5] J. Zhao et al., SonicBOOM: the 3rd generation Berkeley out-of-order machine, CARRV 2020. [6] A. Amid et al., Chipyard: integrated design, simulation and implementation framework, IEEE Micro 2020. [7] CHIA: an open framework for agentic HW/SW co-design flows, <https://chialoops.ai>. [8] RISC-V International, Spike ISA simulator.